Code Representation 101

Guillermo Polito













Goals

- Understand different ways to represent and manipulate code
  - trees, lists, graphs
- Typical instructions

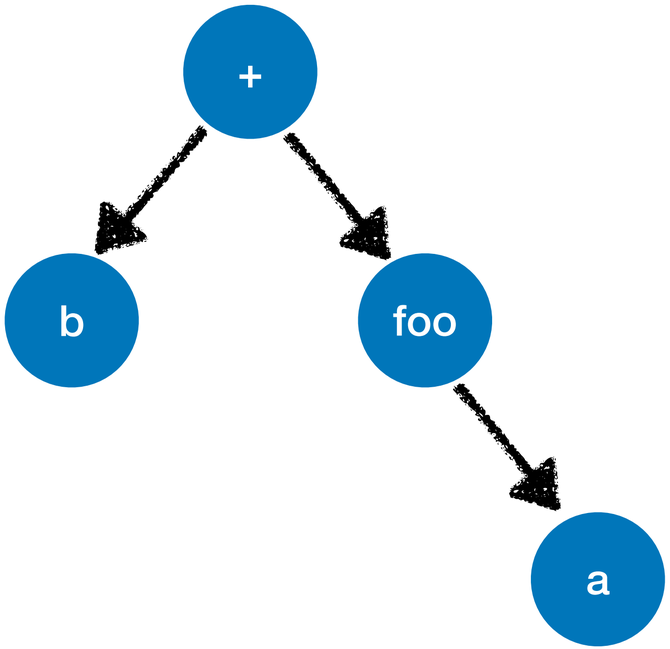
Data structures to represent code

```
b + foo(a);
```

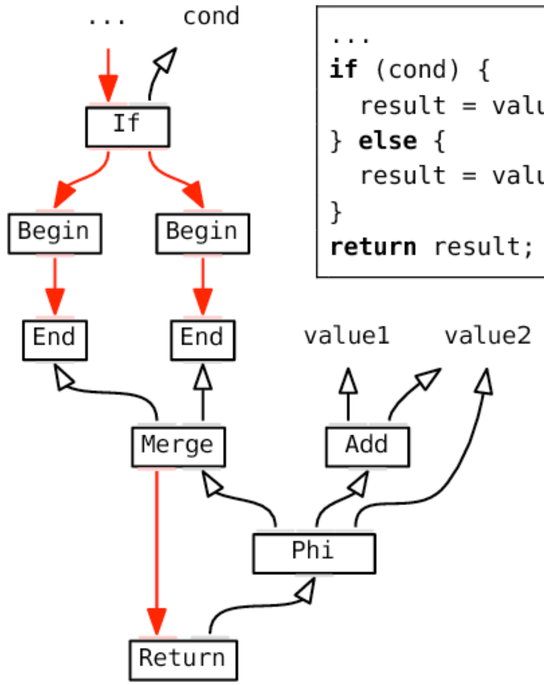
- Lists

LOAD R1, b
MOV R2, R1
LOAD R1, a
CALL FOO
ADD R1, R1, b

- Trees



- DAGs




```
...
if (cond) {
  result = value1 + value2;
} else {
  result = value2;
}
return result;
```

Lists representing code

- Closer to “machine” code
- Simple to manipulate
- Relations between instructions become implicit
  - e.g., how many arguments does `foo` have?
  - e.g., Answer => sometimes, we need to see foo’s code
  - These become “conventions"

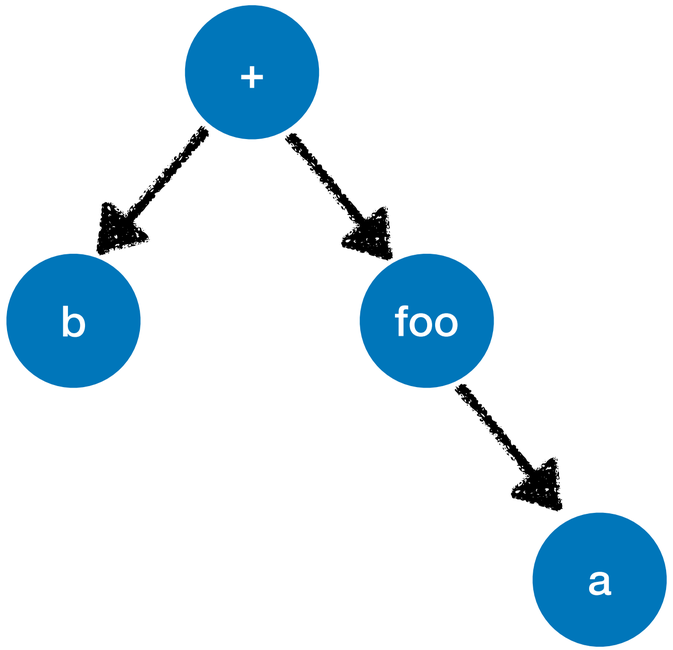
```
b + foo(a);
```

LOAD R1, b
MOV R2, R1
LOAD R1, a
CALL foo
ADD R1, R1, b

Trees representing code

- Closer to source code
- Often produced by a parser
- Relations are explicit
  - e.g., how many arguments does foo have?
  - e.g., Answer => look at foo's children!

```
b + foo(a);
```



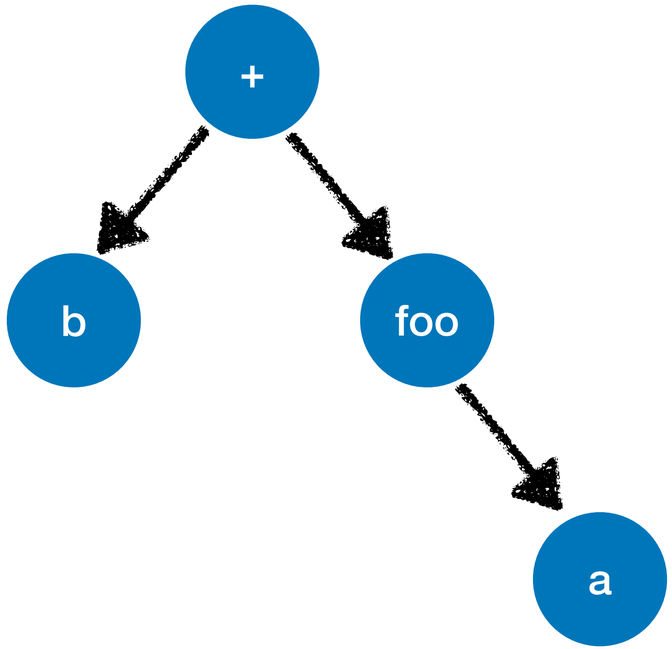
Abstract Syntax Trees (ASTs)

- Trees representing code
- Abstract, because they do not represent ALL elements in a grammar
  - i.e., parentheses, statement finalizers, indentation are **not** in the tree

```
b + foo(a);

b + (foo(a));

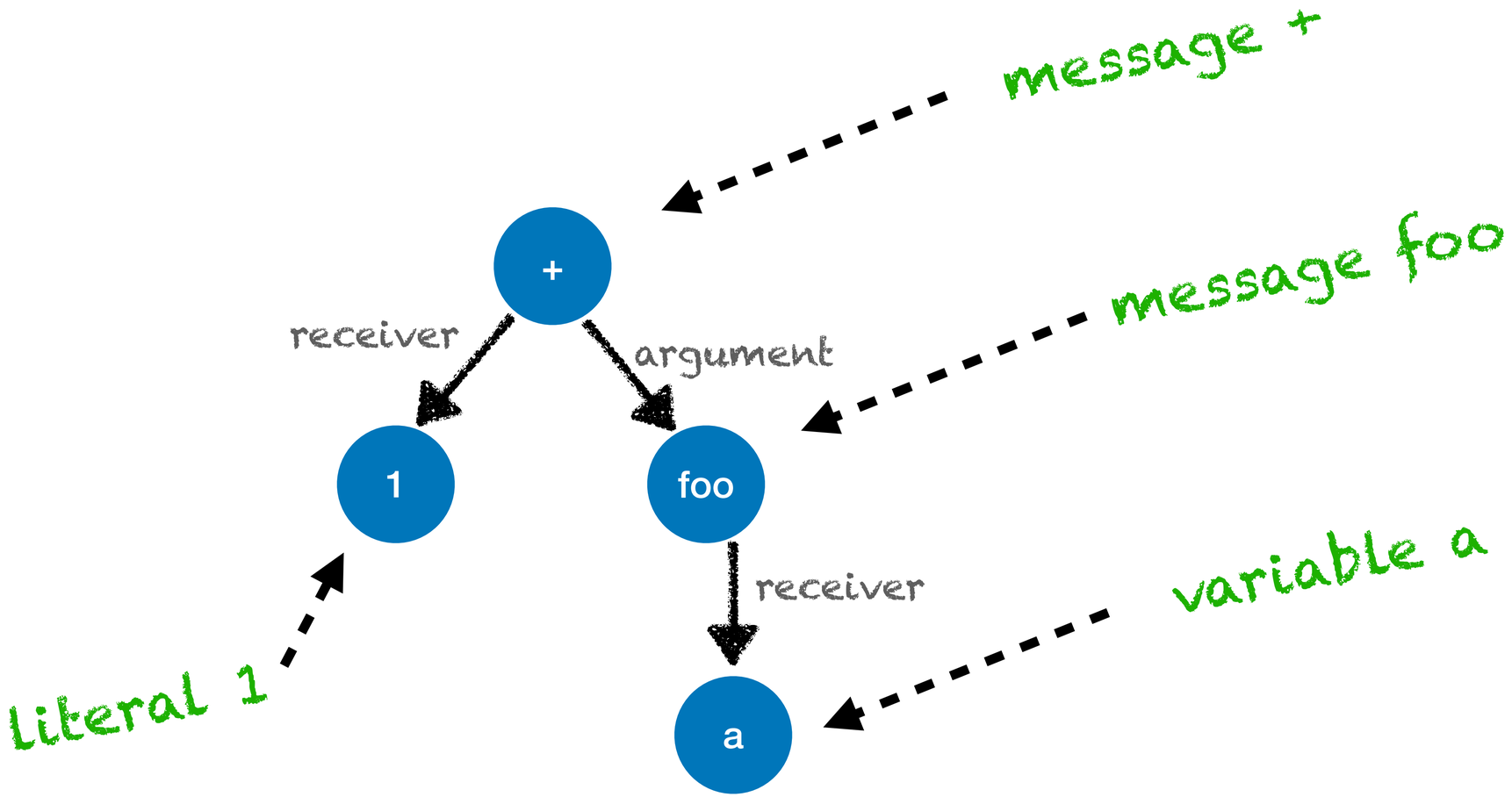
b +
  foo(a)
```



Pharo AST's

Example

```
1 + a foo
```

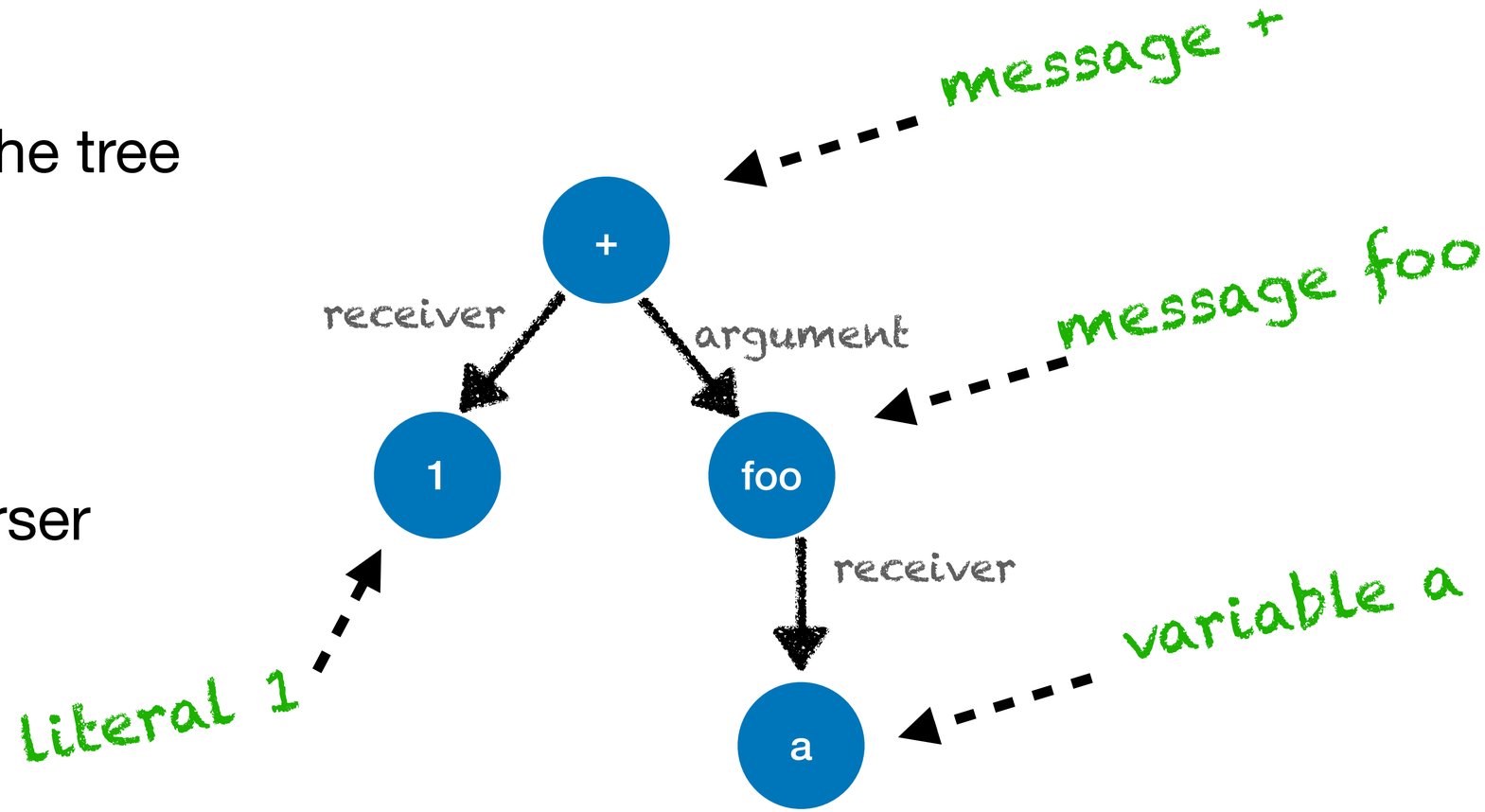


Pharo AST's

Precedence is in the tree!

- Executed first => lower in the tree
- Unary < binary < keyword
- Thus unary is lower
- Already resolved by the parser

```
1 + a foo
```

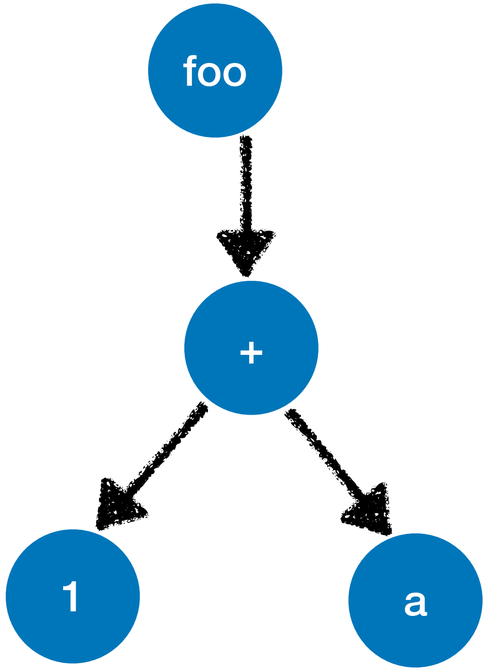


Pharo AST's

Precedence is in the tree, example 2

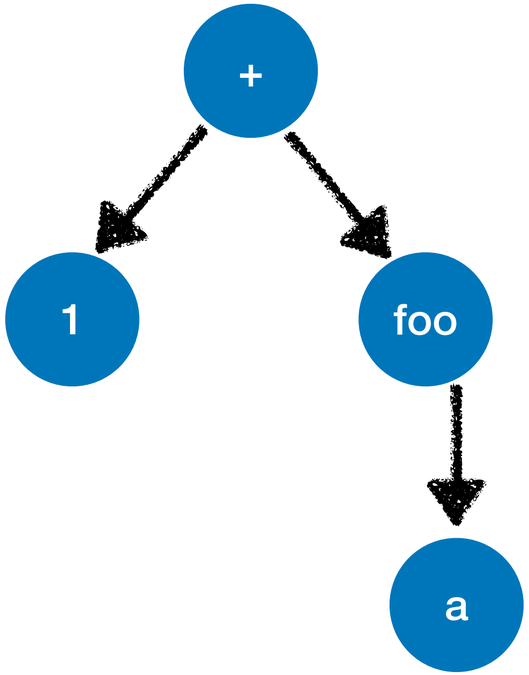
- Executed first => lower in the tree
- Parenthesis < unary < binary < keyword
- Thus parenthesis is lower
- Already resolved by the parser

```
(1 + a) foo
```

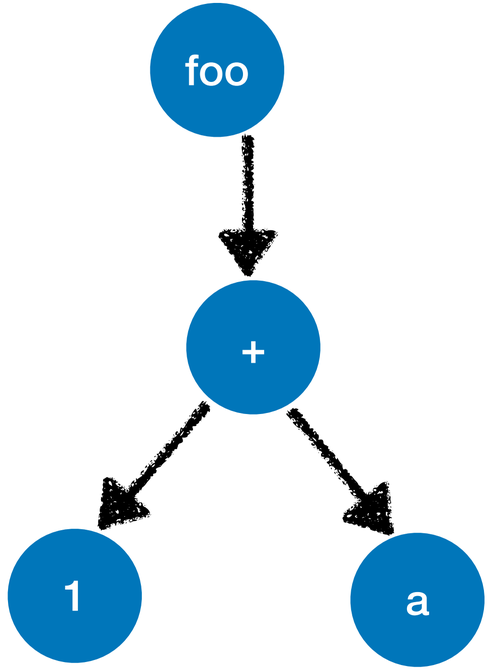


Comparing Precedence

```
1 + a foo
```




```
(1 + a) foo
```



Other Code Representation Flavours

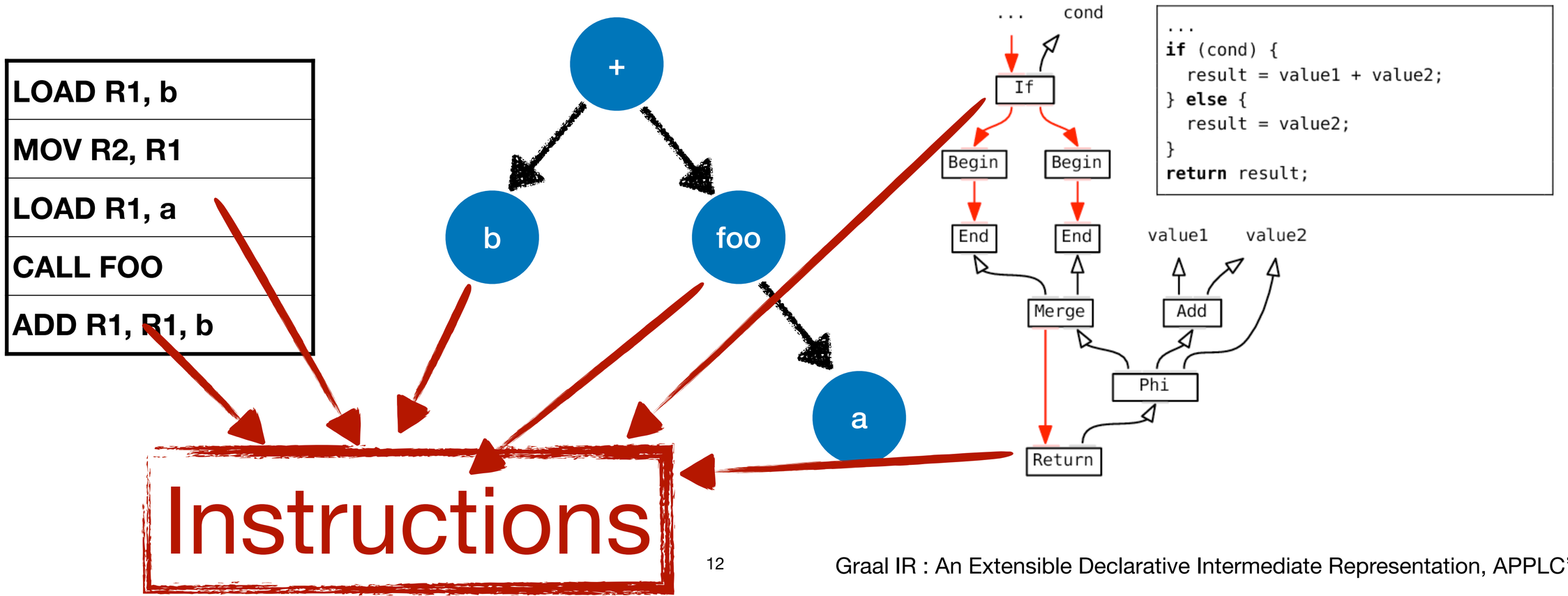
- Control Flow Graphs
  - Nodes represent uninterrupted sequences of instructions
  - Edges represent control-flow
- Sea of nodes
  - Flat representation: control flow and instructions are at the same level
- Single Static Assignment (SSA) form + variants
  - Each variable has max one assignment

Another Day

All need to represent instructions

```
b + foo(a);
```

- Lists
- Trees
- DAGs



Graal IR : An Extensible Declarative Intermediate Representation, APPLC'13

Instruction Set Depends on the Machine

- Basic instructions:
  - Write data to memory
  - Read data from memory
  - Arithmetic (+ , - , * …)
  - Bit instructions (shift, bitAnd …)
  - Control flow (jump, jump if)

Instruction Set Depends on the Machine

- Basic instructions:
  - Write data to memory
  - Read data from memory
  - Arithmetic (+ , - , * …)
  - Bit instructions (shift, bitAnd …)
  - Control flow (jump, jump if)
- Machine-specific instructions:
  - Messages/method invocations
  - Casts and type-checks
  - Memory allocation
  - Garbage collection
  - …

Takeaways

- Compilers are organized in Phases
- Compilers represent code using Intermediate Representations (IRs)
- Compilers are designed with different IRs to achieve different trade-offs
- Phases either
  - modify some code representation e.g., optimizations
  - change some code representation into another e.g., a parser

Material

- Books
  - Engineering a Compiler - IR design

